Visiyon AI

Administrator Manual

Self-hosted AI chat platform — setup, day-to-day management, and troubleshooting

Version 1.1

Table of Contents

1. Overview

Visiyon AI is a self-hosted AI chat platform built on top of Ollama. It provides a chat interface for end users, an admin dashboard for managing models, users and platform settings, and a set of built-in tools (file generation, image generation, document search, web browsing) that the AI can call on a user's behalf.

The platform consists of the following components, each running as its own Docker container:

- **frontend** — the Next.js web application (chat UI and admin dashboard)
- **backend** — the API server (Fastify) that talks to Ollama, the database, and external providers
- **postgres** — the database, storing chats, messages, users, documents and settings
- **nginx** — reverse proxy in front of the frontend and backend

2. Admin Dashboard

The admin dashboard is available to users in the Admin group and covers:

- Model management (display names, visibility, generation parameters)
- Pulling and deleting Ollama models
- User accounts, groups and per-group model permissions
- Document library oversight
- Analytics (usage by user, by model, over time)
- Security alerts and system logs
- Server health (Ollama status, uptime, memory, GPU)

The top bar of the dashboard shows notifications (bell icon), this manual (help icon), and the current admin account (top right).

3. Managing Models

The Models page lists every chat-capable model detected in Ollama. For each model, an admin can:

- Give it a friendly display name shown in the model picker instead of the raw Ollama tag (pencil icon)
- Hide a model from the picker without removing it from Ollama (eye icon) — useful for models kept installed for testing but not meant for everyday use
- Regenerate a model's cached settings (circular arrow icon)
- Adjust default generation parameters — temperature, top_p, stop sequences, etc. (gear icon)

> Renaming or hiding a model only changes what is displayed in Visiyon AI — the underlying model in Ollama is never modified or renamed.

Embedding models are hidden automatically

Models pulled purely for document search (for example nomic-embed-text) cannot hold a conversation and are automatically excluded from both the end-user model picker and the Models management page. They still appear on the Pull/Delete Models page, since an admin may need to manage or remove them there.

4. Pulling and Deleting Models

New models can be pulled directly from the admin dashboard without needing shell access to the server. Pulling a model downloads it to every configured Ollama instance, so it is available regardless of which backend a chat request is routed to.

- Enter the exact Ollama model tag (e.g. llama3.1:8b) and start the pull
- Progress is shown while the download and installation complete
- Deleting a model removes it from every configured Ollama instance

Tip: model tags follow the name:tag format used by Ollama, e.g. qwen2.5:7b or llava:latest.

5. Users, Groups and Permissions

Every account belongs to a group. Groups control which models a user is allowed to see and use. Admin accounts always see every model regardless of group restrictions.

- Create or edit groups and assign an explicit list of allowed models, or leave a group unrestricted
- Move users between groups from the Users page
- Reset a user's chat memory (long-term personalization) from their user detail page, or a user can manage their own memory from Settings > Memory

6. Documents (RAG library)

Users can upload documents (PDF, Word, plain text, Markdown, CSV) to their personal library. Each document is split into chunks and embedded so the AI can search it for relevant context during a conversation — this is called Retrieval-Augmented Generation (RAG).

- Supported types: PDF, DOCX, TXT, Markdown, CSV
- A per-user total storage quota applies (default 500 MB), configurable via the DOCUMENT_USER_QUOTA_BYTES environment variable
- Documents are automatically deleted after a configurable number of days (default 3), via DOCUMENT_MAX_AGE_DAYS — this also removes their chunks and embeddings
- Users can also delete a document manually at any time

7. Generated Images and Files

Generated images

When a user asks the AI to create an image, the result is embedded directly into the chat message. Images uploaded by users to vision-capable models are stored the same way.

Generated files

When a user asks the AI to produce a downloadable file (for example an HTML page or a script), the file is written to disk and served through a temporary, tokenized download link — the file is never guessable or public without that link.

8. Storage Retention and Cleanup

Visiyon AI runs an automatic cleanup job so the server does not fill up with old content over time. Everything below runs on its own without admin action, and every retention period is configurable through environment variables in docker-compose.yml.

Content	Default retention	Environment variable
Generated / uploaded images	30 days	`IMAGE_RETENTION_DAYS`
Uploaded documents (RAG)	3 days	`DOCUMENT_MAX_AGE_DAYS`
Generated downloadable files	24 hours	`n/a (fixed)`
Quota usage history	90 days	`QUOTA_USAGE_RETENTION_DAYS`
Webhook delivery logs	30 days	`WEBHOOK_DELIVERY_RETENTION_DAYS`

Setting a retention variable to 0 disables that particular cleanup and keeps content indefinitely. Changes to these variables require rebuilding and restarting the backend container:

```
cd ~/visiyon
docker compose build backend
docker compose up -d backend
```

Cleaning up existing content immediately

The scheduled job only clears content once it passes its retention age. To clear everything that already exists right now (for example right after lowering a retention period), run the one-off command below against the database, then reclaim the freed disk space with VACUUM:

```
docker exec -it visiyon-postgres psql -U visiyon -d visiyon -c "
UPDATE \"Message\" SET content = regexp_replace(
  content,
  '!\[[^\]]*\]\([A-Za-z0-9+/=]+\)',
```

```
  '![Image expired](about:blank)', 'g'
)
WHERE content LIKE '%data:image%';
UPDATE \"Message\" SET images = '{}' WHERE array_length(images, 1) > 0;
"

docker exec -it visiyon-postgres psql -U visiyon -d visiyon -c "VACUUM FULL \"Message\";"
```

Run VACUUM as its own command — it cannot run inside the same transaction as other statements.

9. Analytics

The Analytics section reports on platform usage over a selectable time range (default 30 days):

- **Summary** — total messages, active users, and overall activity
- **By model** — which models are used most
- **By user** — usage broken down per account
- **Timeseries** — activity trend over time
- **By function/tool** — how often built-in tools (file generation, image generation, web search, web browsing) are used

10. Security Alerts and Logs

The Security section surfaces automatically-flagged events (for example repeated failed logins or unusual access patterns) that may need admin attention. Alerts can be reviewed and marked as resolved. The Logs page shows recent system log entries for troubleshooting.

11. Server-level Maintenance

Restarting services

```
cd ~/visiyon
docker compose restart backend
docker compose restart frontend
docker compose restart nginx
```

Rebuilding after a code change

```
cd ~/visiyon
docker compose build backend  # or: frontend
docker compose up -d backend  # or: frontend
```

Viewing logs

```
docker compose logs backend --tail=80 -f
```

Server time and timezone

Visiyon AI uses the host server's system clock for all timestamps. To set the timezone (example: Amsterdam):

```
sudo timedatectl set-timezone Europe/Amsterdam
sudo timedatectl set-ntp true
timedatectl status
```

12. GPU Passthrough

Giving the backend container direct GPU access lets Admin > Server health show live GPU stats (via nvidia-smi running inside the container). Most deployments don't need this — Ollama and the optional self-hosted image-generation container usually own the GPU on their own, separate from the backend. Only apply this override if the backend container itself has a GPU available on the same host.

Enabling GPU passthrough

```
cd ~/visiyon
docker compose -f docker-compose.yml -f docker-compose.gpu.yml up -d --build
```

This applies docker-compose.gpu.yml on top of the base compose file. Two passthrough mechanisms are declared together in that file, so it works whichever one your host actually has set up:

- The modern Compose syntax (`deploy.resources.reservations.devices`) — only takes effect on Docker Compose v2.3 or newer; older versions silently ignore it, which is the most common reason GPU passthrough "does nothing" with no error
- The legacy mechanism (`runtime: nvidia`) — works on effectively every Compose version, but requires the host's NVIDIA Container Toolkit to be configured as an available Docker runtime first

```
sudo nvidia-ctk runtime configure --runtime=docker
sudo systemctl restart docker
```

Verifying passthrough independently

If GPU stats still don't show up after applying the override, verify passthrough independently of Visiyon AI first:

```
docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi
```

If that command fails too, the issue is the host's NVIDIA Container Toolkit setup, not the Visiyon AI configuration.

To target a specific GPU instead of all of them, replace `count: all` in docker-compose.gpu.yml with `device_ids: ["0"]` (or whichever index `nvidia-smi -L` shows on the host).

Keeping GPU passthrough across auto-updates

A manual `docker compose up` always needs both `-f` flags — that's inherent to how Compose override files work, and no code change can avoid it. Self-updates triggered from the admin panel, however, can pick this up automatically once the following environment variable is set in .env:

```
grep COMPOSE_GPU_OVERRIDE ~/visiyon/.env
```

If it's empty or missing, add it:

```
echo "COMPOSE_GPU_OVERRIDE=docker-compose.gpu.yml" >> ~/visiyon/.env
```

Then restart once more so the updater container also picks up the variable:

```
docker compose -f docker-compose.yml -f docker-compose.gpu.yml up -d
```

From this point on, the updater's own docker compose calls automatically include the GPU override file, so a self-update from the admin panel no longer silently drops GPU passthrough back to a no-GPU state.

13. Troubleshooting

A reply stops mid-generation and disappears

This can happen if the connection to the model drops while a long reply (for example a large generated file) is still streaming. Visiyon AI keeps whatever was already generated instead of discarding it, and closes the reply out cleanly rather than leaving it hanging.

A reply appears to hang with no response

If the model connection stalls silently (no error, no more content), the backend and the browser both give up after 60 seconds of silence and surface a clear, recoverable error instead of waiting indefinitely.

A long reply appears all at once instead of streaming

Ordinary code containing curly braces (CSS rules, JavaScript objects) can be briefly mistaken for the start of a tool call. Visiyon AI checks the characters after any '{' — ignoring whitespace differences — and, if they do not match a genuine tool call, immediately resumes normal live streaming instead of buffering the rest of the reply silently.

Container keeps restarting after a deploy

Check the container logs first — this almost always points directly at the cause:

```
docker compose logs backend --tail=80
```

Where to get help

For deeper issues, check the backend logs as shown above, and confirm the relevant containers were rebuilt (not just restarted) after any code change, since a restart alone reuses the previously built image.